

3강 실습

# Python 기초 문법 실습 세션

변수 · 자료형 · 리스트 · 딕셔너리 · 조건문 · 반복문 · 함수

총 60분 | 7개 주제 × 2~3문제 + 종합 미션

# 실습 안내 — 오늘 60분 동안 이것을 합니다

## ① 변수

2문제 · 6분

## ② 자료형

2문제 · 6분

## ③ 리스트

3문제 · 9분

## ④ 딕셔너리

3문제 · 9분

## ⑤ 조건문

2문제 · 8분

## ⑥ 반복문

2문제 · 8분

## ⑦ 함수

2문제 · 8분

## 종합 미션

2문제 · 12분

💡 \_\_\_\_ 빈칸을 채워서 실행해보세요 | 일찍 끝나면 ☆ 도전 문제에 도전! | 막히면 손 들어주세요 🙋

# 변수 (Variable) — 실습

① 변수

연습 1/2

🕒 3분

내 정보를 변수에 저장하고 출력하세요.

만들어야 할 변수:

- name → 자신의 이름 (문자열)
- age → 나이 (정수)
- height → 키 (소수, 예: 170.5)
- is\_student → 학생 여부 (True/False)

네 변수 모두 print()로 출력하세요.

💡 힌트: 문자열은 "" 또는 ''로 감싸기 / = 는 저장(대입) 연산자

출력 예) 이름: 홍길동   나이: 20   키: 170.5   학생: True

```
# -- 변수 저장하기 -----  
name      = "___"   # 이름 입력  
age        = ___     # 나이 (정수)  
height     = ___     # 키 (소수 가능)  
is_student = ___     # True 또는 False
```

```
# -- 출력하기 -----  
print("이름:", ___)  
print("나이:", ___)  
print("키:", ___)  
print("학생:", ___)
```

```
# 도전: f-string으로 한 줄에 출력해보세요  
# print(f"나는 {___}이고 {___}살입니다.")
```

# 변수 (Variable) — 실습

① 변수

연습 2/2

🕒 3분

변수 연산과 재할당을 연습합니다.

①  $x = 10$  으로 시작

→ 5 증가 후 출력

→ 2배 후 출력

② 두 문자열 변수 연결하기

③ 도전: 두 변수 값 서로 교환!

temp 변수를 활용하세요.

💡 힌트:  $x = x + 5 \rightarrow x$ 에 5를 더해서 다시  $x$ 에 저장

# ① 변수 재할당

```
x = 10
```

```
print("처음 x:", x)
```

```
x = x + _____ # 5 증가
```

```
print("5 더한 x:", x)
```

```
x = x * _____ # 2배
```

```
print("2배 x:", x)
```

# ② 문자열 연결

```
first = "데이터"
```

```
second = "분석"
```

```
result = first + " " + _____
```

```
print(result) # → 데이터 분석
```

# ③ 도전! 두 값 교환하기

```
a, b = 100, 200
```

```
temp = _____
```

```
a = _____
```

```
b = temp
```

```
print(a, b) # → 200 100
```

# 자료형 (Data Type) — 실습

② 자료형

연습 1/2

🕒 3분

type() 함수로 자료형을 확인하고  
왼쪽 표의 빈칸을 채워보세요.

실행해서 맞는지 확인하세요!

▶ 예상 결과를 써보세요

type(42) → \_\_\_\_

type(3.14) → \_\_\_\_

type("hello") → \_\_\_\_

type(True) → \_\_\_\_

type(42 + 0.5) → \_\_\_\_

type("42") → \_\_\_\_

# 자료형 확인

```
print(type(42))  
print(type(3.14))  
print(type("hello"))  
print(type(True))
```

# 연산 결과의 자료형은?

```
print(type(42 + 0.5))    # int + float = ?  
print(type("42"))       # 숫자처럼 보이지만?
```

# 비교해보기

```
print(42 == "42")        # ?  
print(type(42) == type("42")) # ?
```

# 도전: 아래 type을 예상해보세요

```
print(type(None))  
print(type([1, 2, 3]))  
print(type({"key": "value"}))
```

# 자료형 (Data Type) — 실습

② 자료형

연습 2/2

🕒 3분

자료형 변환 함수를 완성하세요.

int() / float() / str() / bool()

중 알맞은 함수를 \_\_\_\_ 에 채우세요.

⚠ 주의: int("3.14") → 오류!  
float("3.14") 이 먼저!

💡 힌트: input()으로 받은 값은 항상 str → 계산하려면 int() 변환 필수

예) score + 10 → str끼리는 오류, int() 후 연산 가능

# 자료형 변환 연습

# 1) 정수 → 문자열

```
num = 2024
```

```
result = ____ (num)
```

```
print(type(result), result) # str 2024
```

# 2) 문자열 → 정수

```
score_str = "88"
```

```
score = ____ (score_str)
```

```
print(score + 10) # → 98
```

# 3) 실수 → 정수 (소수점 버림)

```
pi = 3.99
```

```
print(____ (pi)) # → 3 (반올림 아님!)
```

# 4) 도전: input()으로 받아 계산

```
age_str = input("나이 입력: ")
```

```
next_year = ____ (age_str) + 1
```

```
print("내년 나이:", next_year)
```

# 리스트 (List) — 실습

③ 리스트

연습 1/3

🕒 3분

fruits 리스트에서 값을 꺼내세요.

- ① 첫 번째 항목 출력
- ② 마지막 항목 (음수 인덱스)
- ③ "포도"를 인덱스로 출력
- ④ 앞 3개만 슬라이싱
- ⑤ 리스트 길이
- ⑥ 도전: 역순 출력[::-1]

💡 힌트: 인덱스는 0부터! 슬라이스 [a:b]는 b 직전까지

```
fruits = ["사과", "바나나", "포도", "딸기", "오렌지"]
```

# ① 첫 번째 항목 (인덱스 0)

```
print(fruits[___])      # → 사과
```

# ② 마지막 항목 (음수 인덱스)

```
print(fruits[___])      # → 오렌지
```

# ③ "포도" 출력 (인덱스 2)

```
print(fruits[___])      # → 포도
```

# ④ 앞 3개만 슬라이싱

```
print(fruits[___:___])
```

```
# → ['사과', '바나나', '포도']
```

# ⑤ 리스트 길이

```
print(len(___))         # → 5
```

# 도전: 역순으로 출력

```
print(fruits[___ : ___ : ___])
```

# 리스트 (List) — 실습

③ 리스트

연습 2/3

🕒 3분

scores 리스트 메서드를 완성하세요.

- ① 88 추가 (맨 끝)
- ② 오름차순 정렬
- ③ 최고·최저·합계 출력
- ④ 68 삭제
- ⑤ 평균 계산

💡 힌트: `append` / `sort` / `max` / `min` / `sum` / `remove`

```
scores = [85, 72, 90, 68, 95]
print("원래:", scores)
```

```
# ① 88 추가 (맨 끝에)
scores.__(88)
print("추가 후:", scores)
```

```
# ② 오름차순 정렬
scores.__()
print("정렬:", scores)
```

```
# ③ 통계 출력
print("최고:", ____(scores))
print("최저:", ____(scores))
print("합계:", ____(scores))
```

```
# ④ 68 삭제
scores.__(68)
print("삭제 후:", scores)
```

```
# ⑤ 평균
print("평균:", round(____(scores)/____(scores), 1))
```

# 리스트 (List) — 실습

③ 리스트

연습 3/3

🕒 3분

☆ 응용

grades 에서 조건 맞는 값만 모으기  
(앞으로 자주 쓰는 핵심 패턴!)

① 80점 이상만 high\_scores 에 추가  
빈 리스트 → for문 → if문 → append

② 전체 평균 계산

③ 도전: 평균 이상인 점수만 모아보기

💡 힌트: 빈 리스트 = [] → for + if + append() 로 조건부 추가

```
grades = [78, 85, 92, 65, 88, 74, 91, 83]
```

# ① 80점 이상만 모으기

```
high_scores = [] # 빈 리스트 준비
```

```
for score in grades:
    if score >= __: # 80 이상 조건
        high_scores.__(score)
```

```
print("80점 이상:", high_scores)
print("인원 수:", len(high_scores))
```

# ② 전체 평균

```
avg = __ (grades) / __ (grades)
print("전체 평균:", round(avg, 1))
```

# 도전: 평균 이상인 점수만 모아보세요

```
above_avg = [s for s in grades if s >= __]
print("평균 이상:", above_avg)
```

# 딕셔너리 (Dictionary) — 실습

④ 딕셔너리

연습 1/3

🕒 3분

student 딕셔너리를 완성하세요.

- ① "이름" 값 출력
- ② "GPA" 값 출력
- ③ "학년"을 3으로 수정
- ④ "이메일" 새로 추가
- ⑤ 전체 출력 + 키 목록

💡 힌트: 읽기: dict["키"] / 수정·추가: dict["키"] = 새값

```
student = {
    "이름": "홍길동",
    "학번": "20240001",
    "학과": "컴퓨터공학과",
    "학년": 2,
    "GPA": 3.7
}

# ① 이름 출력
print("이름:", student[____])

# ② GPA 출력
print("GPA:", student[____])

# ③ 학년 수정 (2 → 3)
student[____] = ____

# ④ 이메일 추가
student[____] = "hong@university.ac.kr"

# ⑤ 전체 출력
print(student)
print("키 목록:", list(student.keys()))
```

# 딕셔너리 (Dictionary) — 실습

④ 딕셔너리

연습 2/3

🕒 3분

inventory 딕셔너리 메서드를 완성하세요.

- ① keys()로 과일 목록 출력
- ② values()로 수량 목록 출력
- ③ items() + for문으로 "과일: N개" 출력
- ④ get()으로 없는 키 안전하게 접근
- ⑤ 수량 10개 미만 과일 찾기

💡 힌트: keys() / values() / items() / get(키, 기본값) 활용

```
inventory = {
    "사과": 30, "바나나": 15,
    "포도": 8,  "딸기": 22
}

# ① 과일 목록 (keys)
print(list(inventory.___()))

# ② 수량 목록 (values)
print(list(inventory.___()))

# ③ "과일: N개" 출력 (items + for)
for fruit, count in inventory.___():
    print(f"{fruit}: {count}개")

# ④ 없는 키 안전하게 조회
print(inventory.___("수박", 0))  # → 0

# ⑤ 10개 미만 과일
for fruit, count in inventory.items():
    if count < ___:
        print(f"재고 부족: {fruit}({count}개)")
```

# 딕셔너리 (Dictionary) — 실습

④ 딕셔너리

연습 3/3

🕒 3분

★ 응용

scores 딕셔너리로 성적 분석!

① values()로 전체 평균

② 90점 이상 과목 출력

③ 최고점 과목 찾기

max(dict, key=dict.get)

도전: 최저점 과목도 찾기!

💡 힌트: `sum(dict.values()) / max(dict, key=dict.get)`

```
scores = {  
    "국어": 85, "영어": 92,  
    "수학": 78, "과학": 88, "사회": 74  
}
```

# ① 전체 평균

```
total = sum(scores.values())  
avg = total / len(scores)  
print("평균:", round(avg, 1))
```

# ② 90점 이상 과목

```
print("우수 과목:")  
for subject, score in scores.items():  
    if score >= 90:  
        print(f"{subject}: {score}점")
```

# ③ 최고점 과목

```
best = max(scores, key=scores.get)  
print("최고:", best, scores[best], "점")
```

# 도전: 최저점 과목

```
worst = min(scores, key=scores.get)  
print("최저:", worst)
```

# 조건문 (if / elif / else) — 실습

⑤ 조건문

연습 1/2

🕒 4분

학점 계산기를 완성하세요.

점수 범위 → 학점:

90 이상 → A / 80 이상 → B

70 이상 → C / 60 이상 → D

그 외 → F

60점 이상이면 "합격"

미만이면 "불합격"도 추가!

💡 힌트: elif 순서 중요! 위에서부터 처음 맞는 조건만 실행됨

```
score = int(input("점수를 입력하세요 (0-100): "))
```

```
# 학점 판별 조건 채우기
```

```
if score >= ____:
```

```
    grade = "A"
```

```
elif score >= ____:
```

```
    grade = "B"
```

```
elif score >= ____:
```

```
    grade = "C"
```

```
elif score >= ____:
```

```
    grade = "D"
```

```
else:
```

```
    grade = "F"
```

```
print(f"점수: {score}점, 학점: {grade}")
```

```
# 합격/불합격 판별
```

```
if score >= ____:
```

```
    status = "합격"
```

```
else:
```

```
    status = "불합격"
```

```
print(f"결과: {status}")
```

# 조건문 (if / elif / else) — 실습

⑤ 조건문

연습 2/2

🕒 4분

☆ 응용

BMI 판별기 — 복합 조건 연습

BMI = 몸무게 / (키(m))<sup>2</sup>

18.5 미만 → 저체중

18.5 이상 25 미만 → 정상

25 이상 → 과체중

and 조건을 활용하세요!

💡 힌트: cm를 m로: `height_m = height / 100` / `**` 는 거듭제곱

```
weight = float(input("몸무게(kg): "))
height = float(input("키(cm): "))
```

```
# cm → m 변환
height_m = height / ____
```

```
# BMI 계산
bmi = ____ / (____ ** 2)
print(f"BMI: {round(bmi, 1)}")
```

```
# 복합 조건으로 판별
if bmi < ____:
    status = "저체중"
elif bmi >= ____ and bmi < ____:
    status = "정상"
else:
    status = "과체중"
```

```
print(f"판정: {status}")
```

```
# 도전: 정상이고 20대이면 "매우 건강함!" 출력
age = int(input("나이: "))
if status == "정상" and ____ <= age < ____:
    print("매우 건강함!")
```

# 반복문 (for / while) — 실습

⑥ 반복문

연습 1/2

🕒 4분

for문 기본 패턴을 연습합니다.

- ① 학생 목록을 번호 포함 출력  
enumerate() 활용
- ② 1부터 10까지 합계  
range() 활용
- ③ 도전: 구구단 3단 출력

💡 힌트: enumerate(리스트) → (인덱스, 값) 쌍 / range(1,11) → 1~10

예) 1번: Alice   2번: Bob   ...   1~10합계: 55

```
students = ["Alice", "Bob", "Charlie", "Diana", "Eve"]
```

```
# ① 번호 포함 출력
```

```
for i, name in enumerate(students):  
    print(f"__+1} 번: {name}")
```

```
print("---")
```

```
# ② 1~10 합계
```

```
total = 0  
for i in range(__, __):     # 1부터 10까지  
    total = total + i  
print("1~10 합계:", total)   # → 55
```

```
print("---")
```

```
# 도전: 구구단 3단
```

```
for n in range(1, __):  
    print(f"3 × {n} = {3*n}")
```

# 반복문 (for / while) — 실습

⑥ 반복문

연습 2/2

🕒 4분

☆ 응용

for문 + 조건문 조합 — 학점 분류기

scores 리스트를 순회하면서  
각 점수의 학점을 grades 딕셔너리에  
누적해서 카운트하세요.

마지막에 A학점 학생 번호 출력!

💡 힌트: `grades["A"] += 1` → A 카운터 1 증가

```
scores = [78, 85, 92, 65, 88, 74, 91, 83, 69, 95]
grades = {"A": 0, "B": 0, "C": 0, "F": 0}
```

# 학점 분류 (조건 채우기)

```
for score in scores:
    if score >= __:
        grades[___] += 1
    elif score >= __:
        grades[___] += 1
    elif score >= __:
        grades[___] += 1
    else:
        grades[___] += 1
```

```
print("학점 분포:", grades)
```

# 평균 계산

```
avg = ____(scores) / ____ (scores)
print(f"평균: {round(avg, 1)}점")
```

# 도전: A학점 학생 번호 출력

```
for i, score in enumerate(scores):
    if score >= 90:
        print(f" {i+1}번 학생: {score}점")
```

# 함수 (Function) — 실습

⑦ 함수

연습 1/2

🕒 4분

함수 정의와 호출 연습

- ① greet(name): 인사 출력 함수
- ② add(a, b): 합계 반환 함수
- ③ my\_abs(x): 절댓값 함수  
abs() 쓰지 말고 if문으로!

도전: greet에 기본값 추가  
def greet(name, title="님")

💡 힌트: def 함수명(매개변수): → return 반환값 / 출력은 print()

```
# ① 인사 함수
def greet(name):
    print(____)    # "안녕하세요, {name}님!" 출력

greet("홍길동")    # → 안녕하세요, 홍길동님!
greet("Python")    # → 안녕하세요, Python님!

# ② 합계 함수 (return)
def add(a, b):
    return ____

result = add(10, 25)
print("합계:", result)    # → 35

# ③ 절댓값 직접 구현 (abs() 금지!)
def my_abs(x):
    if x >= 0:
        return ____
    else:
        return ____

print(my_abs(-7))    # → 7
print(my_abs( 3))    # → 3
```

# 함수 (Function) — 실습

⑦ 함수

연습 2/2

🕒 4분

☆ 응용

성적 처리 함수 만들기

① calc\_avg(scores): 평균 반환

② get\_grade(score): 학점 반환

두 함수를 조합해서

my\_scores의 평균과 학점 출력

도전: 최고·최저 제외 평균 함수!

💡 힌트: sorted(리스트)[1:-1] → 최솟값·최댓값 제외한 새 리스트

# ① 평균 계산 함수

```
def calc_avg(scores):  
    return ____ (scores) / ____ (scores)
```

# ② 학점 판별 함수

```
def get_grade(score):  
    if score >= 90: return "A"  
    elif score >= 80: return "B"  
    elif score >= ____: return "C"  
    else: return "F"
```

# 함수 활용하기

```
my_scores = [85, 92, 78, 90, 88]
```

```
avg = calc_avg(____)  
print(f"평균: {round(avg, 1)}")  
print(f"학점: {get_grade(avg)}")
```

# 도전: 최고·최저 제외 평균

```
def trimmed_avg(scores):  
    trimmed = sorted(scores)[____:____]  
    return calc_avg(trimmed)
```

```
print(f"조정평균: {round(trimmed_avg(my_scores), 1)}")
```

# 🌀 종합 미션 1 — 학생 성적 분석기

🕒 6분

변수 + 리스트 + 딕셔너리 + 함수 종합

학생 리스트를 순회하며  
각 학생의 평균·학점을 출력하세요.

딕셔너리 키: "name", "scores"

출력 형식:

"김민준: 평균 85.0점, B학점"

〈 예상 출력 〉

김민준: 평균 85.0점, B학점  
이서연: 평균 91.0점, A학점  
박지호: 평균 71.7점, C학점  
정하은: 평균 91.0점, A학점

```
students = [
    {"name": "김민준", "scores": [85, 92, 78]},
    {"name": "이서연", "scores": [90, 88, 95]},
    {"name": "박지호", "scores": [72, 68, 75]},
    {"name": "정하은", "scores": [88, 94, 91]}
]

def calc_avg(scores):
    return sum(scores) / len(scores)

def get_grade(avg):
    if avg >= 90: return "A"
    elif avg >= 80: return "B"
    elif avg >= 70: return "C"
    else: return "F"

for student in students:
    name = student["name"]
    scores = student["scores"]
    avg = calc_avg(scores)
    grade = get_grade(avg)
    print(f"{name}: 평균 {round(avg, 1)}점, {grade}학점")
```

## 🌀 종합 미션 2 — 숫자 통계 프로그램

🕒 6분

변수 + 리스트 + 반복문 + 조건문 종합

숫자를 5개 입력받아 통계 출력

input() + int() + append() + for문  
+ sum / max / min / len 을 활용!

마지막에 평균 이상인 숫자만 출력

💡 힌트: 빈 리스트에 int(input(...)) 값을 append()로 추가

< 예상 출력 (입력: 10 25 8 30 15) >

```
합계: 88   평균: 17.6
최댓값: 30   최솟값: 8
평균 이상: [25, 30]
```

```
# 숫자 5개 입력받기
numbers = []

for i in range(5):          # 5번 반복
    num = int(input(f"{i+1} 번째 숫자: "))
    numbers.append(num)      # 리스트에 추가

# 통계 출력
print("\n=== 결과 ===")
print("숫자 목록:", numbers)
print("합계:", sum(numbers))
avg = sum(numbers) / len(numbers)
print(f"평균: {round(avg, 1)}")
print("최댓값:", max(numbers))
print("최솟값:", min(numbers))

# 평균 이상인 숫자만 출력
above = []
for n in numbers:
    if n >= avg:
        above.append(n)
print("평균 이상:", above)
```

# ☆ 도전 미션 — 나만의 성적 관리 프로그램

자유 도전

일찍 끝난 분께 추천!

## 🎯 목표

과목과 점수를 입력받아  
나만의 성적부를 만드세요.

## 📋 구현 조건

- ① 과목 수 입력
- ② 과목명 + 점수 입력 (for문)
- ③ 딕셔너리에 저장
- ④ 전체 평균 계산
- ⑤ 최고·최저 과목 출력
- ⑥ 학점별 과목 분류

💡 정해진 정답 없음!  
자유롭게 만들어보세요.

```
# ☆ 나만의 성적 관리 프로그램
# 아래를 참고해서 자유롭게 완성하세요!

n = int(input("과목 수를 입력하세요: "))
gradebook = {}

for i in range(n):
    subject = input(f"과목 {i+1} 이름: ")
    score = int(input(f"{subject} 점수: "))
    gradebook[subject] = score

print("\n=== 나의 성적표 ===")
for subject, score in gradebook.items():
    print(f" {subject}: {score}점")

# 여기서부터 직접 구현해보세요!
# ④ 전체 평균:

# ⑤ 최고/최저 과목:

# ⑥ 학점 분류:
```

# 수고하셨습니다!

✓ 변수 & 자료형 — 저장과 변환

✓ 리스트 & 딕셔너리 — 데이터 구조

✓ 조건문 & 반복문 — 흐름 제어

✓ 함수 — 재사용 가능한 코드

## 다음 시간 : pandas로 실제 데이터 분석 시작!

오늘 배운 7개 개념이 pandas · 시각화 · 머신러닝의 기반이 됩니다.  
모르는 부분은 오늘 안에 다시 복습해주세요!